// Day 8 of 10

Day 8

10 System Design Concepts

Developers make these every day.
Don't be one of them.

→ Swipe through all 10

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #01

Horizontal vs Vertical Scaling

Know when to use each

```
// Bad
// Just keep adding more RAM/CPU
// Vertical has hard limits!
// Single point of failure

// Good
Vertical: Bigger server (simpler)
Horizontal: More servers (resilient)
// Prefer horizontal for web apps
```

💡 Horizontal = resilient, Vertical = simpler

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #02

Load Balancer

Distributes traffic across servers

```
// Bad
// One server handles everything
// It goes down = entire app down!

// Good
Client → Load Balancer
         ├── Server 1
         ├── Server 2
         └── Server 3
// Round-robin distribution
```

💡 LB is the front door of scaled apps

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #03

Caching Strategy

Don't hit DB for same data repeatedly

```
// Bad
// Every request queries database
// Same data fetched 1000x/minute!

// Good
Check Cache → HIT? Return fast
              MISS? → DB → Cache → Return
// Redis / Memcached
```

💡 Cache what's read often, changes rarely

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #04

CDN for Static Assets

Serve from nearest location

```
// Bad
// Images/JS/CSS from your server
// User in US, server in India = slow

// Good
// CDN caches at edge nodes worldwide
// User gets data from nearest node
// Cloudflare, AWS CloudFront
```

💡 CDN = speed + reduced server load

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #05

Database Indexing

Indexes speed up reads dramatically

```
// Bad
SELECT * FROM users
WHERE email = 'x@test.com';
// Full table scan = O(n)!

// Good
CREATE INDEX idx_email ON users(email);
// Now O(log n) lookup
// Critical for tables > 100k rows
```

💡 Index columns used in WHERE/JOIN/ORDER

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #06

Message Queues

Decouple services, handle spikes

```
// Bad
// Order placed → send email synchronously
// Email slow = order API slow!

// Good
Order placed → Queue (Kafka/RabbitMQ)
             → Consumer sends email async
// API fast, email handled separately
```

💡 Queues = async, resilient, decoupled

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #07

Database Sharding

Split large DB across multiple nodes

```
// Bad
// 1 billion users, 1 database
// Too big, too slow!

// Good
Shard 1: users 0–10M
Shard 2: users 10M–20M
Shard 3: users 20M–30M
// Each shard = smaller, faster
```

💡 Shard when single DB can't scale

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #08

CAP Theorem

Distributed systems choose 2 of 3

```
// Bad
// Thinking you can have ALL THREE:
// Consistency + Availability
// + Partition Tolerance

// Good
CP: MongoDB — consistent + partition-safe
AP: Cassandra — available + partition-safe
// Choose based on your needs
```

💡 Know your system's CAP trade-off

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #09

Rate Limiting

Protect your API from abuse

```
// Bad
// No limits at all
// Bot sends 100k req/sec
// Server crashes

// Good
// Token bucket / sliding window
100 requests per minute per IP
// Return 429 Too Many Requests
```

💡 Rate limit = stability + security

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #10

SQL vs NoSQL

Pick the right tool for the job

```
// Bad
// Using MongoDB for everything
// or MySQL for everything
// One size doesn't fit all!

// Good
SQL: Relational, ACID, complex queries
NoSQL: Scale, flexibility, simple access
// Choose based on data shape
```

💡 Choose based on data shape & scale needs

Shubham Bhati
linkedin.com/in/bhatishubham

```
done = true
```

Follow for more tips!

Like · Comment · Repost

Shubham Bhati
linkedin.com/in/bhatishubham